

Semantic Search

Multimedia Retrieval · Chapter 5

Multimedia Retrieval - 2026 - Uni Basel

Contents

5 - Semantic Search	1
Introduction	2
The Evolution of Semantic Representations	2
How This Chapter Is Organized	3
Latent Semantic Indexing	4
Singular Value Decomposition	4
Application in Text Retrieval	5
Worked Example	6
Limitations of LSI	7
Word Embeddings	8
From LSI to Embeddings	8
Word2Vec	8
GloVe and Subword Models	10
Emergent Properties and Limitations	10
Transformer Embeddings	12
The Attention Mechanism	12
BERT: Bidirectional Context	13
Bi-Encoders: Sentence-BERT	13
Cross-Encoders and Reranking	14
Late Interaction: ColBERT	15
Building Semantic Search	17
Embedding Model Selection	17
Document Chunking and Encoding	18
Retrieval Architectures	18
Efficiency at Scale	19
Practical Considerations	20
Summary	21
Model Comparison	21
Key Takeaways	21
Key Formulas	22
Self-Check Questions	22
Further Reading	23

5 - Semantic Search

Classical retrieval matches keywords; semantic search matches meaning. This chapter develops the full progression from matrix factorization methods that first revealed latent structure in text, through the neural revolution of word embeddings and transformers, to the production-ready retrieval pipelines deployed in modern search systems. By the end, you will understand how to design a system that retrieves documents about “cardiac conditions” when a user searches for “heart disease”, even when no terms overlap.

Introduction

A user searching for “heart disease” expects to find documents about myocardial infarction, coronary artery conditions, and cardiology treatment, even when those documents share no keywords with the query. Classical retrieval models based on BM25 or the vector space model treat terms as independent symbols: they can match “heart” to “heart” but not to “cardiac”. Stemming and lemmatization help at the surface level, and hand-curated ontologies can map synonyms, but these approaches are labor-intensive, language-specific, and brittle when facing the full diversity of natural language expression.

Semantic search solves this problem by representing text as dense vectors in a continuous space where geometric proximity encodes meaning. Two words that never co-occur in a query-document pair can still be recognized as related if their embeddings point in similar directions. This shift from discrete term matching to continuous meaning matching is the central theme of this chapter.

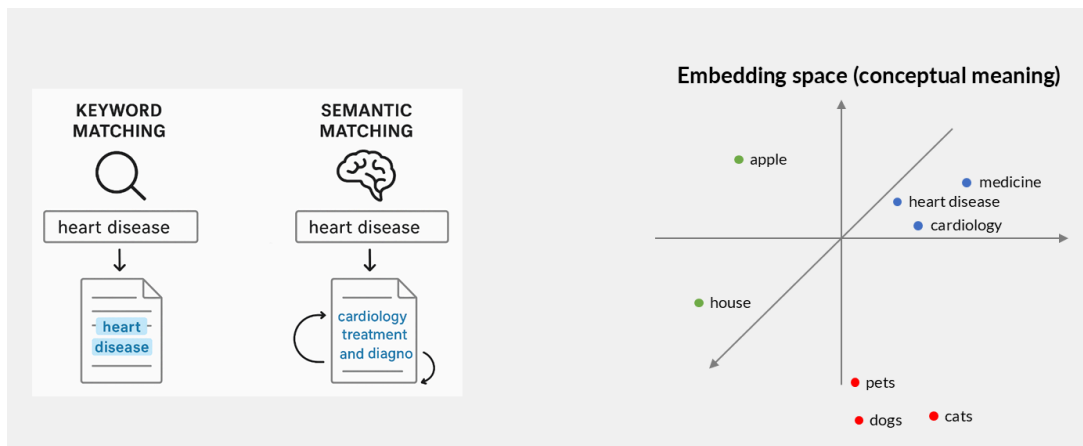


Figure 5.1: Keyword matching operates on exact surface forms (left). Semantic matching maps terms into a continuous embedding space where conceptually related terms cluster together (right), enabling retrieval even when query and document share no words.

The Evolution of Semantic Representations

The path from keyword retrieval to modern semantic search spans three decades of research, each generation addressing limitations of its predecessor:

1. **Latent Semantic Indexing** (Deerwester et al., 1988) applied Singular Value Decomposition to the term-document matrix, discovering latent topics that link related terms. LSI proved that dimensionality reduction could capture semantic relationships, but its computational cost and inability to use inverted indices limited practical adoption.
2. **Static word embeddings** (Mikolov et al., 2013; Pennington et al., 2014) learned dense vector representations from local context windows. Word2Vec and GloVe showed that simple neural architectures trained on large corpora produce vectors where semantic similarity corresponds to geometric proximity. However, each word receives a single vector regardless of context: “bank” has the same representation whether it refers to a financial institution or a riverbank.
3. **Contextual embeddings and transformers** (Vaswani et al., 2017; Devlin et al., 2019; Reimers and Gurevych, 2019) introduced attention mechanisms that produce different representations for the same word depending on its surrounding context. Sentence-BERT restructured transformers into efficient bi-encoders that generate high-quality sentence embeddings suitable for large-scale retrieval.
4. **Modern retrieval architectures** (Khattab and Zaharia, 2020; Kusupati et al., 2022) introduced late interaction models like ColBERT that balance quality and efficiency, Matryoshka embeddings that enable multi-resolution search, and hybrid pipelines that combine sparse and dense retrieval for robustness.



Figure 5.2: The modern retrieval pipeline: a fast retriever selects candidate documents from the index, then a re-ranker applies deeper semantic analysis to produce the final ranked list.

How This Chapter Is Organized

This chapter covers the full progression from early matrix methods to production-ready semantic search systems:

- **Latent Semantic Indexing** introduces dimensionality reduction via SVD and shows how LSI discovers latent topics that connect related terms.
- **Word Embeddings** presents Word2Vec, GloVe, and subword models that learn distributional word representations from context.
- **Transformer Embeddings** explains the attention mechanism, BERT, bi-encoders (SBERT), cross-encoders, and late interaction models (ColBERT).
- **Building Semantic Search** covers practical pipeline design: embedding model selection, document chunking, hybrid retrieval, and efficiency techniques.

The next chapter (**Vector Search**) addresses the indexing problem: how to efficiently search through millions of embedding vectors using approximate nearest neighbor algorithms.

Latent Semantic Indexing

In classical retrieval, documents are sparse high-dimensional vectors with TF-IDF weights, and similarity depends on shared terms. This term independence assumption limits recall: a document about “automobile maintenance” will not match a query for “car repair” unless we explicitly expand the query with synonyms. Stemming and lemmatization reduce some mismatches, but they are language-specific, require manual effort, and cannot capture domain-specific relationships. A search for “tokens” in a text processing context should also retrieve paragraphs discussing “terms” or “words”, but no stemmer will make that connection.

Latent Semantic Indexing (LSI) addresses this by projecting the high-dimensional term-document space into a compact, lower-dimensional representation where semantically related terms are brought closer together. The key insight is that terms appearing in similar document contexts are likely related, even if they never co-occur in the same document. By reducing dimensionality, LSI forces the model to merge these co-occurrence patterns into latent topics.

Singular Value Decomposition

LSI builds on the Singular Value Decomposition (SVD), which factorizes any $m \times n$ matrix into three components. Given a term-document matrix A of rank r , the SVD produces:

$$A = USV^T \quad (5.1)$$

where U is an $m \times r$ orthonormal matrix (columns represent terms in the latent space), S is an $r \times r$ diagonal matrix of singular values in decreasing order, and V is an $n \times r$ orthonormal matrix (columns represent documents in the latent space).

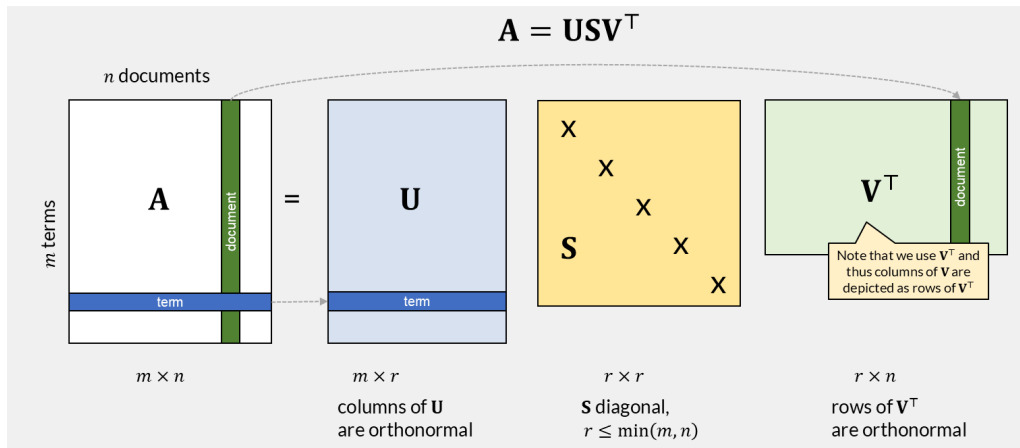


Figure 5.3: SVD decomposes the $m \times n$ term-document matrix A into three factors: U (term representations), S (singular values capturing topic importance), and V^T (document representations).

The singular values on the diagonal of S decrease rapidly in magnitude. By retaining only the k largest singular values and their corresponding columns in U and V , we obtain the best rank- k approximation of A under the Frobenius norm:

$$A_k = U_k S_k V_k^T \quad (5.2)$$

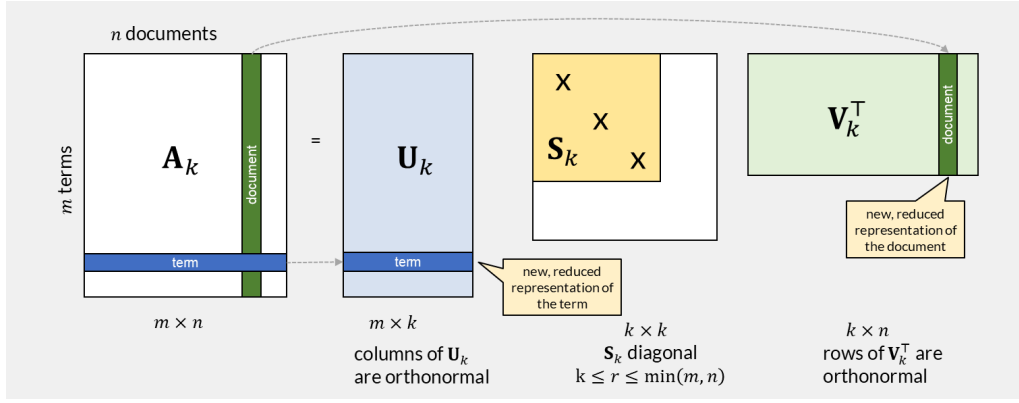


Figure 5.4: Truncated SVD retains only k dimensions. The reduced matrices U_k , S_k , V_k^T provide compact representations: columns of V_k are k -dimensional document vectors, and columns of U_k are k -dimensional term vectors.

The k dimensions no longer correspond to individual terms. Each dimension represents a latent topic, a pattern of term co-occurrence across documents. We can interpret a topic by examining the terms with the largest weights in the corresponding column of U_k .

Application in Text Retrieval

In text retrieval, A is typically the TF-IDF weighted term-document matrix with m vocabulary terms and n documents. After computing the truncated SVD:

- **Document representations:** columns of V_k (equivalently, rows of V_k^T) give k -dimensional vectors for each document.
- **Term representations:** columns of U_k give k -dimensional vectors for each term.
- **Topic interpretation:** the i -th column of U_k describes which terms contribute to the i -th latent topic.

Folding in new documents and queries

When a new document d arrives (represented as a term vector in the original m -dimensional space), we project it into the k -dimensional latent space without recomputing the full SVD:

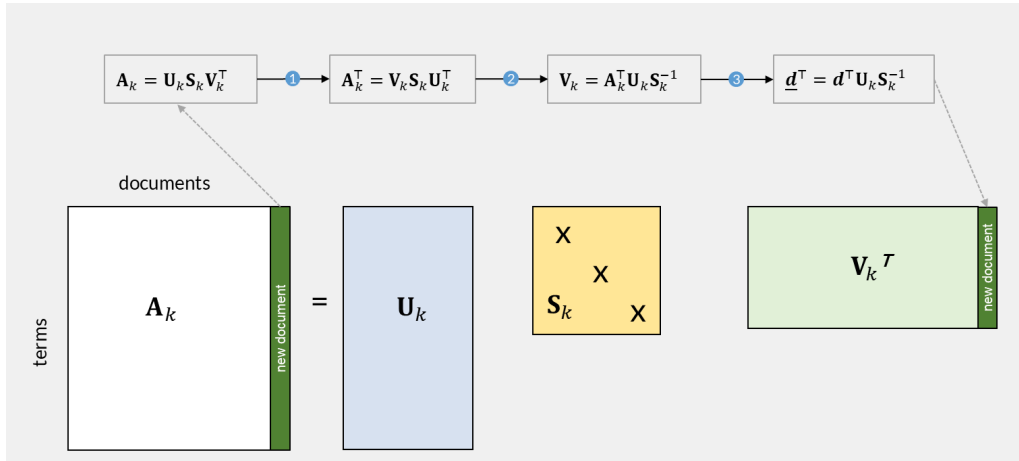


Figure 5.5: Derivation of the folding-in formula in three steps: transpose the truncated SVD equation, multiply by the pseudo-inverse, and isolate a single document vector.

Queries are treated as short documents. The query vector q (with non-zero entries for query terms) is projected identically:

$$\bar{q}^T = q^T U_k S_k^{-1} \quad (5.3)$$

Retrieval then computes cosine similarity between the projected query and all projected documents:

$$\text{sim}_{\cos}(Q, D_i) = \frac{\bar{q} \cdot \bar{d}_i}{|\bar{q}| \cdot |\bar{d}_i|} \quad (5.4)$$

Worked Example

Consider a collection of 9 documents split into two groups: five about human-computer interfaces (c1-c5) and four about graph algorithms (m1-m4):

ID	Title
c1	Human machine interface for Lab ABC computer applications
c2	A survey of user opinion of computer system response time
c3	The EPS user interface management system
c4	System and human system engineering testing of EPS
c5	Relation of user-perceived response time to error measurement
m1	The generation of random, binary, unordered trees
m2	The intersection graph of paths in trees
m3	Graph minors IV: Widths of trees and well-quasi-ordering
m4	Graph minors: A survey

A search for “human computer interaction” should retrieve c1-c5, but not every document contains those exact terms. Traditional retrieval requires explicit query expansion.

After constructing the term-document matrix (using term frequencies, excluding stop words and terms appearing only once), we compute the SVD and truncate to $k = 2$ topics.

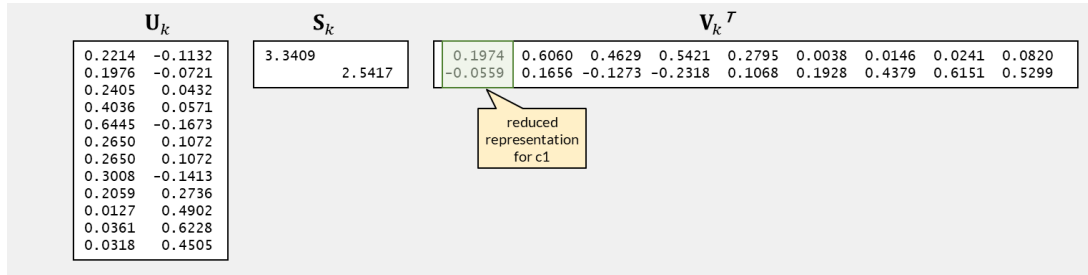


Figure 5.6: Truncated SVD with $k = 2$: the matrices U_k (12 terms $\times$ 2 topics), S_k (2 $\times$ 2), and V_k^T (2 $\times$ 9 documents). Each document is now a 2D vector.

Projecting the query “human computer interaction” (only “human” and “computer” are in the vocabulary) into the 2D topic space yields $\bar{q} = (0.138, -0.028)$ (after normalizing sign conventions).

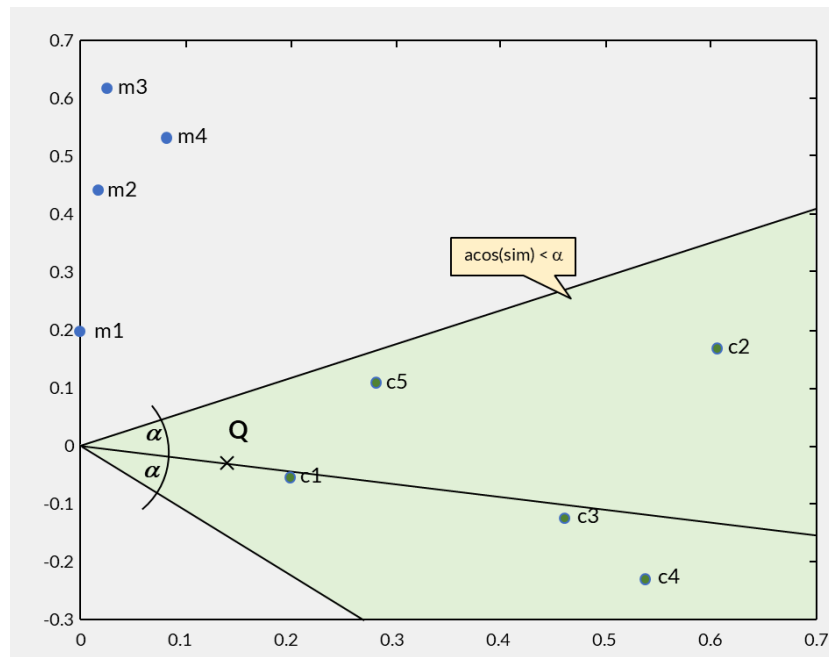


Figure 5.7: Documents and query in the 2D latent space. Topic 1 (horizontal) aligns with the interface documents c1-c5, while topic 2 (vertical) aligns with the graph documents m1-m4. The shaded angular cone shows all documents within angle α of the query vector.

The query vector points toward the c-documents. Using cosine similarity, documents rank as: c3 (0.997) > c1 (0.997) > c4 (0.979) > c2 (0.895) > c5 (0.846), with all c-documents retrieved before any m-document. Notably, c3 (“The EPS user interface management system”) ranks highest despite containing neither “human” nor “computer”. The SVD has recognized that its terms (“EPS”, “user”, “interface”, “system”) co-occur with query-related terms in other documents, connecting them through the latent topic structure.

The two topics can be interpreted from U_k :

- **Topic 1:** $0.64 \cdot \text{system} + 0.40 \cdot \text{user} + 0.30 \cdot \text{eps} + 0.27 \cdot \text{time} + 0.27 \cdot \text{response} + 0.24 \cdot \text{computer}$
- **Topic 2:** $0.62 \cdot \text{graph} + 0.49 \cdot \text{trees} + 0.45 \cdot \text{minors} + 0.27 \cdot \text{survey}$

Limitations of LSI

LSI demonstrated that latent semantic structure improves retrieval, but several limitations prevented widespread adoption:

1. **No inverted index acceleration:** projected document vectors are dense, so every query requires comparison against all documents. The pruning strategies that make inverted indices fast (early termination, skip pointers) do not apply.
2. **Corpus-specific topics:** the learned mapping depends on both terms and documents in the collection. An LSI model trained on IT articles performs poorly on biomedical text. Unlike modern embeddings, LSI representations do not transfer across domains.
3. **Static vocabulary:** new terms can only be approximated through the existing topic structure. Two new terms in the same document receive identical representations, since both depend solely on that document’s projection into V_k .
4. **No sub-word awareness:** LSI operates at the word level and cannot generalize to morphological variants or misspellings not already in the vocabulary.
5. **Linear assumption:** SVD captures linear relationships between terms and documents. Non-linear semantic patterns (metaphor, irony, context-dependent meaning) are beyond its reach.

Despite these limitations, LSI established the foundational principle that underpins all modern semantic search: by reducing dimensionality, we can discover latent structure that connects terms sharing similar usage patterns, even when they never co-occur directly. The methods that follow in this chapter, from Word2Vec to transformers, all build on this insight while addressing LSI’s specific shortcomings.

Word Embeddings

Word embeddings map each word in a vocabulary to a dense vector of d dimensions (typically 100-300), where words appearing in similar contexts receive similar vectors. Unlike LSI, which derives representations from a global term-document matrix, word embeddings learn from local context windows and scale efficiently to billions of training tokens. This section covers the foundational models: Word2Vec, GloVe, and fastText.

From LSI to Embeddings

While both LSI and word embeddings produce dense vector representations, they differ in fundamental ways:

Aspect	LSI	Word Embeddings
Unit of representation	Documents (and terms as by-product)	Words (documents via aggregation)
Context signal	Global co-occurrence across entire documents	Local context windows (typically 5-10 words)
Training	SVD on term-document matrix	Neural network with gradient descent
Vocabulary	Fixed; retraining needed for new terms	Sub-word variants possible (fastText)
Transferability	Corpus-specific; poor cross-domain	Language-level; transfers across collections
Scalability	$O(mnk)$ for SVD; expensive for large corpora	$O(n \cdot d \cdot w)$; scales to web corpora

The key conceptual shift is the **distributional hypothesis**: a word's meaning is defined by the company it keeps (Firth, 1957). Word2Vec operationalizes this by training a neural network to predict either context from a center word (Skip-Gram) or a center word from its context (CBOW).

Word2Vec

Word2Vec, introduced by Mikolov et al. in 2013, maps words to a d -dimensional vector space using self-supervised learning on raw text. No labeled data is required. It has two architectures:

Skip-Gram

The Skip-Gram model operates within a context window of size $2m + 1$ around a center word and learns to predict context words from the center word. For the phrase “the dog chases a cat” with center word “chases” and $m = 2$:

$$P(\text{the, dog, a, cat} \mid \text{chases}) = P(\text{the} \mid \text{chases}) \cdot P(\text{dog} \mid \text{chases}) \cdot P(\text{a} \mid \text{chases}) \cdot P(\text{cat} \mid \text{chases}) \quad (5.5)$$

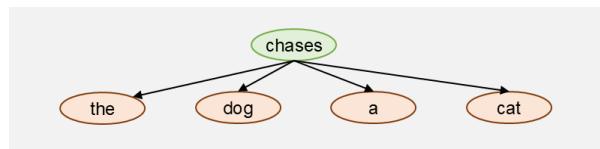


Figure 5.8: Dependency tree for “the dog chases a cat”. In the Skip-Gram model, the center word “chases” predicts each surrounding word independently.

Each word w_i has two d -dimensional vectors: $\mathbf{v}_i$ when used as a center word, and $\mathbf{u}_i$ when used as a context word. The conditional probability uses a softmax over the entire vocabulary $\mathbb{T}$:

$$P(w_s \mid w_c) = \frac{\exp(\mathbf{u}_s^\top \mathbf{v}_c)}{\sum_{i \in \mathbb{T}} \exp(\mathbf{u}_i^\top \mathbf{v}_c)} \quad (5.6)$$

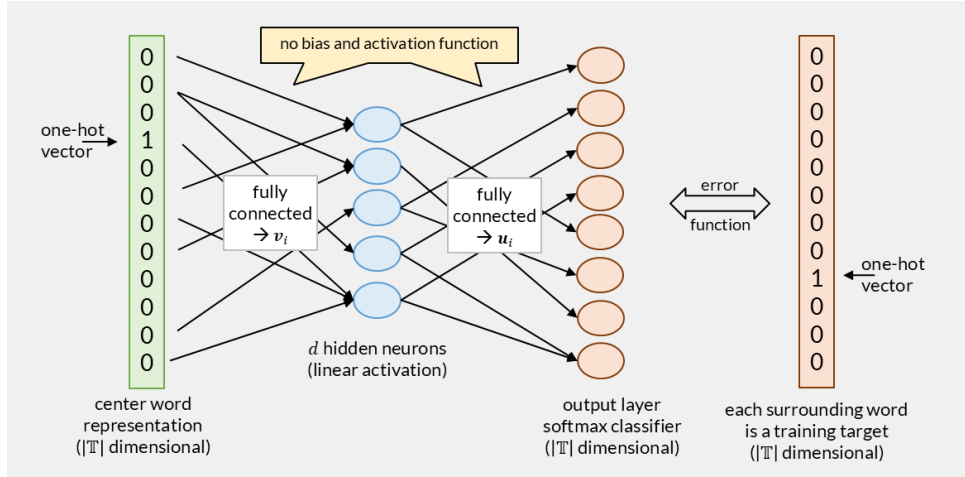


Figure 5.9: Skip-Gram as a neural network: a one-hot input (center word) is projected through a weight matrix (yielding v_i), passed through d hidden neurons with linear activation, then projected to vocabulary size (yielding u_i), and compared to the target context word via softmax and cross-entropy loss.

CBOW

The Continuous Bag of Words (CBOW) model reverses the prediction direction: it predicts the center word from the average of its context word vectors:

$$P(w_c \text{ mid } w_{c-m}, \dots, w_{c+m}) = \frac{\exp(u_c^\top (v_{c-m} + \dots + v_{c+m}))}{\sum_{i \in T} \exp(u_i^\top (v_{c-m} + \dots + v_{c+m}))} \quad (5.7)$$

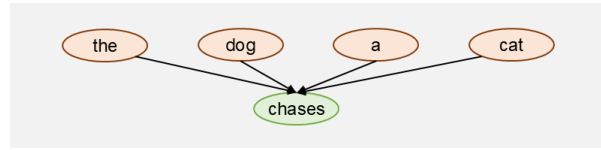


Figure 5.10: In CBOW, all context words point to the center word “chases”, which serves as the prediction target.

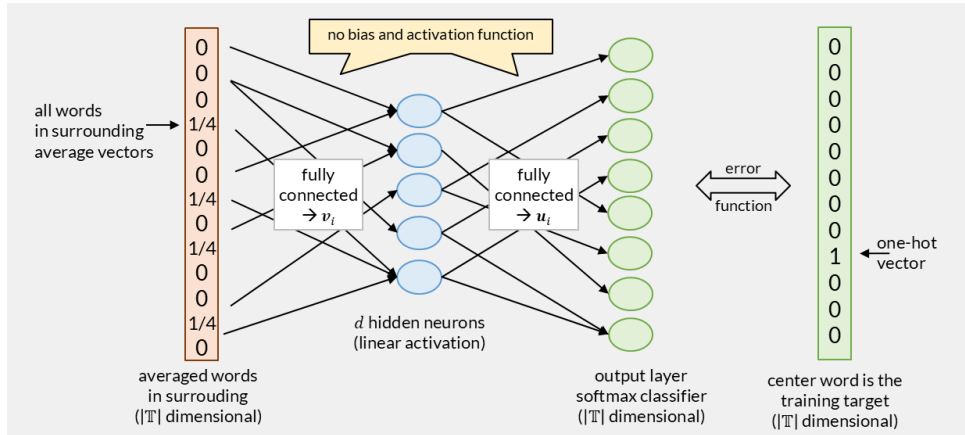


Figure 5.11: CBOW architecture: context word vectors are averaged into a single $|T|$ -dimensional input, passed through a hidden layer of d neurons, then a softmax classifier predicts the center word.

CBOW produces one training sample per window (context around center word), while Skip-Gram produces $2m$ samples (center word around each context word). In practice, Skip-Gram performs better on rare words because it generates more training signal per occurrence, while CBOW trains faster and works well for frequent words.

Training

A PyTorch implementation uses the Embedding layer to avoid explicit one-hot vectors:

```
class SkipGramModel(nn.Module):
    def __init__(self, vocab_size: int, embed_size: int):
        super().__init__()
        self.embeddings = nn.Embedding(vocab_size, embed_size)
        self.linear = nn.Linear(embed_size, vocab_size)

    def forward(self, inputs):
        x = self.embeddings(inputs)
        x = self.linear(x)
        return x
```

Training creates pairs from context windows across the corpus. Sub-sampling frequent words (reducing pairs with stop words to ~1% of their count) improves quality and speed.

GloVe and Subword Models

GloVe

GloVe (Pennington et al., 2014) also learns word vectors from co-occurrence, but uses a global co-occurrence matrix rather than local windows. Let X_{ij} count how often word w_j appears in the context of w_i . GloVe's insight is that the *ratio* of co-occurrence probabilities P_{ik}/P_{jk} distinguishes related from unrelated word pairs better than raw counts.

The model optimizes a weighted least-squares objective:

fastText

fastText (Bojanowski et al., 2017) improves Word2Vec by representing each word as the sum of its sub-word (character n-gram) vectors. For the center word “chases” with 3-grams:

$$\mathbf{v}_{\text{chases}} = \sum_{g \in \mathcal{Z}} \mathbf{z}_g = \mathbf{z}_{<\text{ch}} + \mathbf{z}_{\text{cha}} + \mathbf{z}_{\text{has}} + \mathbf{z}_{\text{ase}} + \mathbf{z}_{\text{ses}} + \mathbf{z}_{\text{es} >} \quad (5.8)$$

This sub-word approach has two advantages:

1. **Morphological generalization:** “chasing”, “chased”, and “chases” share sub-words, so they receive related vectors without explicit stemming.
2. **Handling unknown words:** any new word can be represented by combining its sub-word vectors, even if the word itself was never seen during training.

fastText also introduces **negative sampling** to replace the expensive softmax over the full vocabulary. Instead of normalizing across all $|\mathbb{T}|$ terms, it contrasts the true context pair against m randomly sampled negative words, dramatically reducing computation.

Sub-word lengths can be fixed (3-6 characters) or variable using BPE or WordPiece algorithms, trading vocabulary size against representation granularity.

Emergent Properties and Limitations

Word embeddings trained on large corpora exhibit remarkable emergent structure:

- **Semantic clustering:** words with related meanings form clusters in the embedding space.
- **Analogies:** vector arithmetic captures relational patterns. The classic example: $\mathbf{v}_{\text{Paris}} - \mathbf{v}_{\text{France}} + \mathbf{v}_{\text{Germany}} \approx \mathbf{v}_{\text{Berlin}}$.
- **Odd-one-out detection:** the word least similar to a group can be identified by distance from the group centroid.

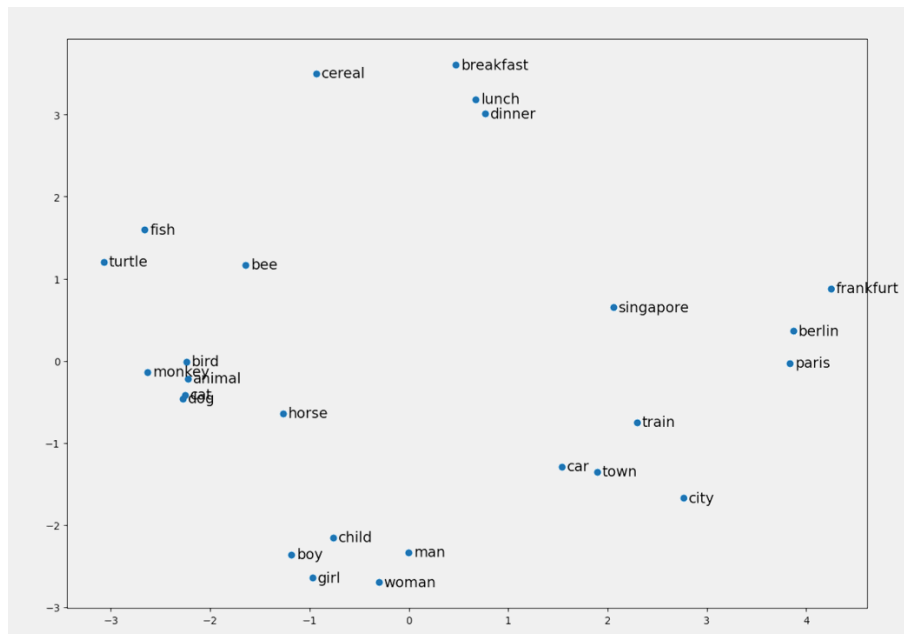


Figure 5.12: PCA projection of GloVe embeddings for 25 words. Semantic clusters emerge: animals (bottom-left), meals (top), cities (right), and people (bottom-center) occupy distinct regions despite no explicit category supervision.

Despite these strengths, static embeddings have a fundamental limitation: **each word receives a single vector regardless of context**. The word "bank" gets the same embedding whether it means a financial institution or a riverbank. In the sentence "I went to the bank on Park Street; I sat and watched the boats go by", humans resolve the ambiguity instantly, but Word2Vec cannot.

This polysemy problem, combined with the lack of word-order sensitivity, motivated the development of contextual embeddings using transformer architectures, covered in the next section.

Transformer Embeddings

Static word embeddings assign a single vector to each word regardless of context. This fails for polysemous words and ignores the compositional meaning that emerges from word combinations. The transformer architecture (Vaswani et al., 2017) solved this by introducing self-attention: a mechanism that lets each token's representation be influenced by every other token in the sequence. This section traces the path from the attention mechanism through BERT to the modern retrieval architectures: bi-encoders, cross-encoders, and late interaction models.

The Attention Mechanism

The core idea of self-attention is that a word's meaning in context depends on its relationships with all other words in the sequence. In "The bank approved the loan", the meaning of "bank" is determined by its relationship with "approved" and "loan". Self-attention computes these relationships explicitly.

For a sequence of n token embeddings, self-attention transforms each token by three steps:

1. **Project** each token embedding x_i into three vectors: a query q_i , a key k_i , and a value v_i using learned weight matrices W_Q, W_K, W_V :

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i \quad (5.9)$$

2. **Score** how much each token should attend to every other token via scaled dot products between queries and keys:

$$\alpha_{ij} = \text{softmax} \left(\frac{q_i^\top k_j}{\sqrt{d_k}} \right) \quad (5.10)$$

3. **Aggregate** the values weighted by attention scores to produce the new representation:

$$z_i = \sum_{j=1}^n \alpha_{ij} v_j \quad (5.11)$$

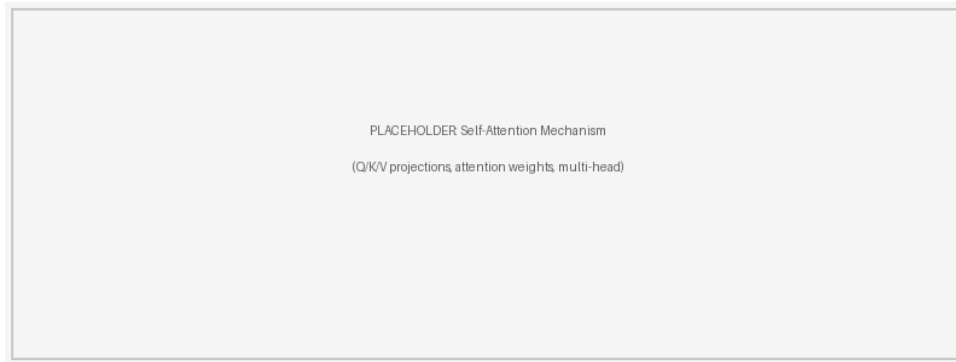


Figure 5.13: [PLACEHOLDER] Self-attention mechanism: each token produces a query, key, and value vector. Attention weights (computed from query-key dot products) determine how much each token's value contributes to the output representation. Multi-head attention runs h parallel attention operations with different projections, then concatenates results.

Multi-head attention runs h parallel attention operations with different projection matrices, allowing the model to attend to different types of relationships simultaneously (syntactic, semantic, positional). The outputs are concatenated and projected:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O \quad (5.12)$$

A transformer encoder stacks multiple layers of multi-head self-attention followed by feed-forward networks. BERT-Base uses 12 layers with 12 attention heads per layer; BERT-Large uses 24 layers with 16 heads.

BERT: Bidirectional Context

BERT (Devlin et al., 2019) applies the transformer encoder to produce contextualized token representations. Unlike GPT, which processes text left-to-right (suitable for generation), BERT attends to context from both directions simultaneously, making it better suited for understanding and representation tasks.

Input construction

BERT processes input through three summed embeddings:

1. **Token embeddings:** WordPiece sub-word units (vocabulary of $\sim 30,000$ tokens). The word “playing” becomes [“play”, “##ing”].
2. **Positional encodings:** learned position vectors that preserve sequence order.
3. **Segment encodings:** markers distinguishing sentence A from sentence B when processing sentence pairs.

Two special tokens frame the input: [CLS] at the start (whose final hidden state serves as a sequence-level representation) and [SEP] separating segments. Sequences are padded to 512 tokens maximum.

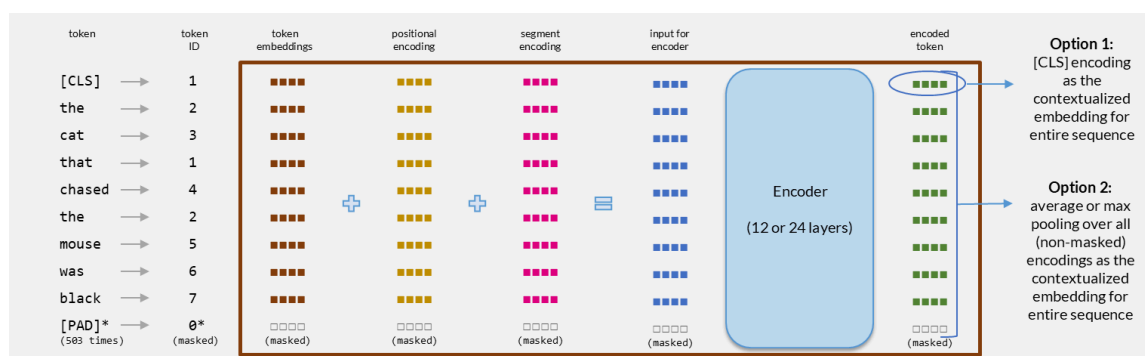


Figure 5.14: BERT input construction for “the cat that chased the mouse was black”. Token embeddings, positional encodings, and segment encodings are summed to form the encoder input. The 12/24-layer encoder produces contextualized output vectors. Option 1: use the [CLS] token encoding as the sequence embedding. Option 2: pool (average or max) over all non-masked token encodings.

Pre-training

BERT is pre-trained on two self-supervised tasks:

- **Masked Language Modeling (MLM):** randomly mask 15% of input tokens and predict them from context.
- **Next Sentence Prediction (NSP):** given two sentences, predict whether the second follows the first in the original text.

After pre-training on large corpora (Wikipedia + BookCorpus), BERT’s encoder produces 768-dimensional (Base) or 1024-dimensional (Large) contextualized vectors for each token. The same word receives different representations depending on its surrounding context.

From BERT to sentence embeddings

Using raw BERT output for retrieval was initially disappointing. The [CLS] token, while designed for classification, produces poor sentence embeddings without task-specific fine-tuning. Mean pooling over token outputs performs marginally better but still underperforms even simple Word2Vec averaging on semantic similarity benchmarks. The breakthrough came with Sentence-BERT.

Bi-Encoders: Sentence-BERT

Sentence-BERT (Reimers and Gurevych, 2019) restructures BERT into a **bi-encoder** (also called dual-encoder or siamese network): two sentences are encoded independently by the same transformer, then compared via cosine similarity.

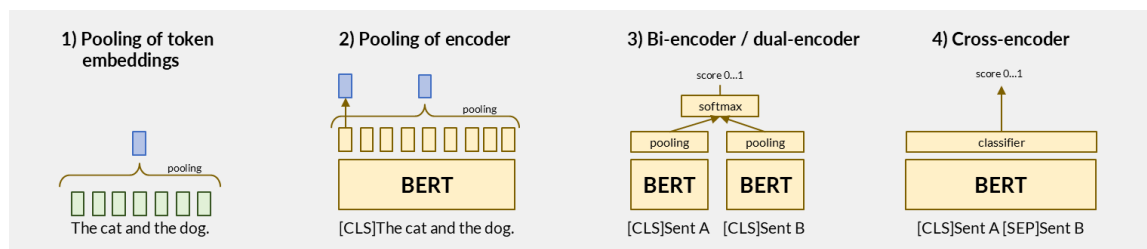


Figure 5.15: Four architectures for sentence similarity, progressing from simple to powerful: (1) pooling static word embeddings, (2) pooling BERT token outputs, (3) bi-encoder with independently encoded sentences compared via learned similarity, (4) cross-encoder that jointly processes both sentences for maximum accuracy.

Architecture and training

The bi-encoder produces fixed-size sentence embeddings:

1. Pass a sentence through BERT (or any transformer encoder).
2. Apply mean pooling over the non-masked token outputs to produce a single d -dimensional vector.
3. Normalize the vector to unit length.

Training uses similarity-based losses on labeled sentence pairs:

- **Contrastive loss:** pulls similar pairs close, pushes dissimilar pairs apart with a margin.
- **Triplet loss:** given an anchor, a positive, and a negative, ensures $\text{sim}(a, p) > \text{sim}(a, n) + \epsilon$.
- **Multiple negatives ranking loss (MNRL):** treats all other examples in the batch as negatives, maximizing efficiency of each training step.

Hard negative mining (selecting negatives that are challenging but incorrect) is critical for training high-quality bi-encoders.

Inference efficiency

The bi-encoder's key advantage for retrieval: documents are encoded once during indexing. At query time, only the query needs encoding. Comparison is a dot product (equivalent to cosine similarity for normalized vectors), enabling sub-millisecond ranking over millions of pre-computed embeddings.

Modern bi-encoder variants

Since SBERT, the field has advanced significantly:

- **Instruction-tuned embeddings:** models like E5-Mistral, GTE-Qwen2, and Nomic Embed accept a task instruction prefix (e.g., "Represent this document for retrieval:") that adapts the embedding to the downstream task without retraining.
- **Decoder-based encoders:** Qwen3-Embedding and similar models build on decoder transformers (using the final [E0S] token representation instead of [CLS]), achieving state-of-the-art quality at the cost of larger models (0.6B to 8B parameters).
- **Extended context:** modern embedding models support 8k-32k token inputs, enabling direct encoding of long documents without chunking.

Cross-Encoders and Reranking

A cross-encoder processes the query and document together as a single input sequence, separated by [SEP]:

[CLS] query [SEP] document [SEP]

The [CLS] token's final hidden state passes through a classification layer to produce a relevance score between 0 and 1. Because all tokens from both query and document attend to each other through every transformer layer, the cross-encoder captures fine-grained token-level interactions that bi-encoders miss.

Quality vs efficiency tradeoff

Property	Bi-Encoder	Cross-Encoder
Encoding	Query and document separately	Query and document jointly

Property	Bi-Encoder	Cross-Encoder
Pre-computation	Documents encoded at index time	Every pair must be processed at query time
Complexity per query	$O(1)$ per document (dot product)	$O(n \cdot L)$ per document (L = sequence length)
Semantic depth	Good (independent representations)	Excellent (full token-level interaction)
Typical use	First-stage retrieval	Reranking top- k candidates

Two-stage pipeline

Cross-encoders are too expensive for exhaustive search over large collections. The standard deployment pattern is:

1. **Retrieve**: a bi-encoder (or BM25) selects the top- k candidates (typically $k = 100 - 1000$).
2. **Rerank**: a cross-encoder scores each candidate with full token interaction and reorders the list.

This hybrid approach achieves near cross-encoder quality at near bi-encoder speed.

Late Interaction: ColBERT

ColBERT (Khattab and Zaharia, 2020) occupies the middle ground between bi-encoders and cross-encoders by computing interaction at the token level while still allowing document pre-computation.

Architecture

Unlike bi-encoders (which compress each text into a single vector) or cross-encoders (which process both texts jointly), ColBERT:

1. **Encodes** query and document independently through a transformer, retaining all token-level embeddings (not pooling to a single vector).
2. **Interacts** via late interaction: each query token computes maximum similarity against all document tokens.
3. **Scores** by summing these maximum similarities across query tokens:

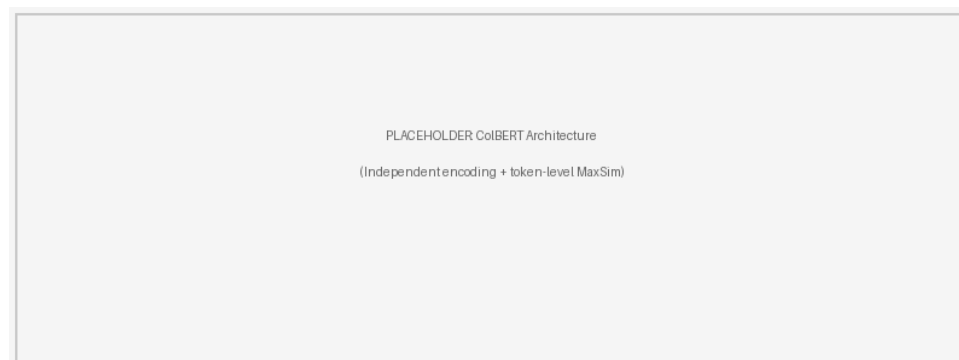


Figure 5.16: [PLACEHOLDER] ColBERT architecture: query and document are encoded independently (allowing document pre-computation), but scoring uses token-level MaxSim interaction rather than a single-vector dot product.

Why late interaction works

The MaxSim operation captures important phenomena that single-vector similarity misses:

- **Exact term matching**: a query token “Python” will have very high similarity with the document token “Python”, directly contributing to the score.
- **Semantic matching**: a query token “automobile” will have high similarity with the document token “car” through their embedding proximity.
- **Partial matching**: not every query token needs to match strongly; the sum allows documents to score well by matching the most important query aspects.

ColBERT v2 and efficiency

ColBERT v2 (Santhanam et al., 2022) addresses the storage cost of retaining per-token embeddings for every document through **residual compression**: token embeddings are quantized by storing only the residual from the nearest centroid, reducing storage by 6-10x while maintaining quality.

The PLAID engine further accelerates retrieval by using centroid interaction for candidate generation before computing full MaxSim only on the top candidates.

Architecture comparison

Architecture	Document representation	Query-time cost	Quality
Bi-encoder	1 vector per document	1 dot product per doc	Good
ColBERT	n vectors per document	\$	q
Cross-encoder	None (must re-encode)	Full transformer pass per doc	Excellent

ColBERT achieves 95-98% of cross-encoder quality with 100-1000x faster query processing, making it practical for direct retrieval (not just reranking) over moderately large collections.

Building Semantic Search

The previous sections presented the models that produce semantic representations. This section addresses the engineering question: how do we assemble these components into a working retrieval system? A production semantic search pipeline must choose an embedding model, decide how to chunk documents, select a retrieval architecture, and manage efficiency at scale.

Embedding Model Selection

The Massive Text Embedding Benchmark (MTEB) provides a standardized evaluation framework for comparing embedding models across multiple tasks: retrieval, semantic textual similarity, classification, clustering, pair classification, and reranking. As of mid-2025, MTEB evaluates models on 56+ datasets across these tasks, with retrieval-specific metrics (nDCG@10) being most relevant for search applications.

Key selection criteria

When choosing an embedding model for a retrieval system, consider:

Criterion	Range	Tradeoff
Embedding dimension	256-4096	Higher dimensions capture more nuance but increase storage and computation
Model size	33M-8B parameters	Larger models produce better embeddings but require more compute for encoding
Context window	512-32k tokens	Longer windows encode more context per chunk but increase encoding time
Multilingual support	1-100+ languages	Multilingual models sacrifice some single-language quality for breadth
Instruction support	Yes/No	Instruction-tuned models adapt to specific tasks via prefixes

Instruction-tuned embeddings

Modern embedding models accept task-specific instructions that adapt the representation without retraining:

```
# For encoding documents
instruction = "Represent this document for retrieval: "
embedding = model.encode(instruction + document_text)

# For encoding queries
instruction = "Represent this query for retrieving relevant documents: "
embedding = model.encode(instruction + query_text)
```

This asymmetric encoding (different instructions for queries vs documents) improves retrieval quality by 3-5% on MTEB benchmarks compared to symmetric encoding.

Model landscape (2025)

Representative models spanning the quality-efficiency spectrum:

Model	Parameters	Dimensions	Context	MTEB Retrieval (nDCG@10)
all-MiniLM-L6-v2	33M	384	512	~49
nomic-embed-text-v1.5	137M	768	8192	~55
jina-embeddings-v3	570M	1024	8192	~58
GTE-Qwen2-7B	7B	3584	32k	~61
Qwen3-Embedding-8B	8B	4096	32k	~63

Note

Benchmark scores evolve rapidly. The specific numbers above reflect mid-2025 results. Always consult the current MTEB leaderboard (<https://huggingface.co/spaces/mteb/leaderboard>) when making production decisions.

Document Chunking and Encoding

Embedding models have finite context windows, and retrieval granularity affects both precision and recall. Documents must be split into chunks that balance:

- **Semantic coherence:** each chunk should represent a complete thought or topic.
- **Size constraints:** chunks must fit within the model's context window.
- **Retrieval granularity:** smaller chunks enable precise matching but may lose context; larger chunks provide more context but dilute the embedding with irrelevant content.

Chunking strategies

Fixed-size chunking splits text at regular token intervals (e.g., every 256 or 512 tokens) with overlap between adjacent chunks (typically 10-20%). Simple to implement but may split sentences or paragraphs mid-thought.

Semantic chunking uses structural cues (paragraph boundaries, section headers, sentence boundaries) to create naturally coherent chunks. More complex but produces chunks aligned with the document's information structure.

Recursive chunking starts with large structural splits (sections), then recursively subdivides any chunk exceeding the target size by progressively finer boundaries (paragraphs, sentences, fixed size).

Encoding pipeline

A typical encoding pipeline for a document collection:

1. **Preprocessing:** extract text, normalize whitespace, optionally remove boilerplate.
2. **Chunking:** split into overlapping or non-overlapping segments.
3. **Metadata attachment:** each chunk retains a reference to its source document, position, and any relevant metadata (title, section heading, date).
4. **Batch encoding:** the embedding model encodes chunks in batches on GPU.
5. **Normalization:** L2-normalize all vectors for dot-product retrieval.
6. **Indexing:** store vectors in a vector database or approximate nearest neighbor index (covered in Chapter 6).

Retrieval Architectures

Three dominant patterns structure modern semantic search pipelines:

Option A: BM25 + semantic reranker

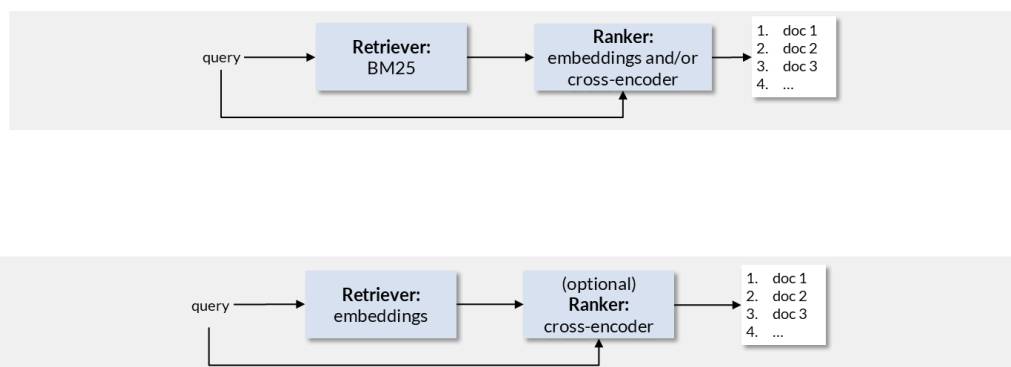


Figure 5.17: Two retrieval pipeline architectures. Top: BM25 retrieves candidates, then an embedding model or cross-encoder reranks them. Bottom: dense embedding retrieval with optional cross-encoder reranking.

BM25 first-stage retrieves the top- k candidates (typically $k = 100 - 1000$) using the inverted index. A **semantic reranker** (cross-encoder or bi-encoder) then reorders these candidates by semantic similarity. This architecture leverages existing infrastructure (inverted indices are well-understood and fast) while adding semantic depth.

Strengths: fast, handles exact-match queries well, low infrastructure cost. Weakness: BM25 recall ceiling limits semantic reranking; if the relevant document is not in the top- k candidates, no reranker can recover it.

Option B: Dense retrieval + optional reranker

A bi-encoder retrieves the top- k documents directly from a vector index using approximate nearest neighbor search. An optional cross-encoder reranks the results for higher precision.

Strengths: captures semantic matches that BM25 misses entirely. Weakness: poor on exact-match queries (proper nouns, codes, IDs); requires vector index infrastructure.

Option C: Hybrid retrieval (sparse + dense)

The most robust modern approach fuses both BM25 and dense retrieval results:

1. BM25 retrieves top- k_1 candidates (handles exact matches, rare terms).
2. Dense retrieval retrieves top- k_2 candidates (handles semantic matches).
3. **Reciprocal Rank Fusion (RRF)** or linear score combination merges both result lists.
4. A cross-encoder reranks the merged top- k for final ordering.

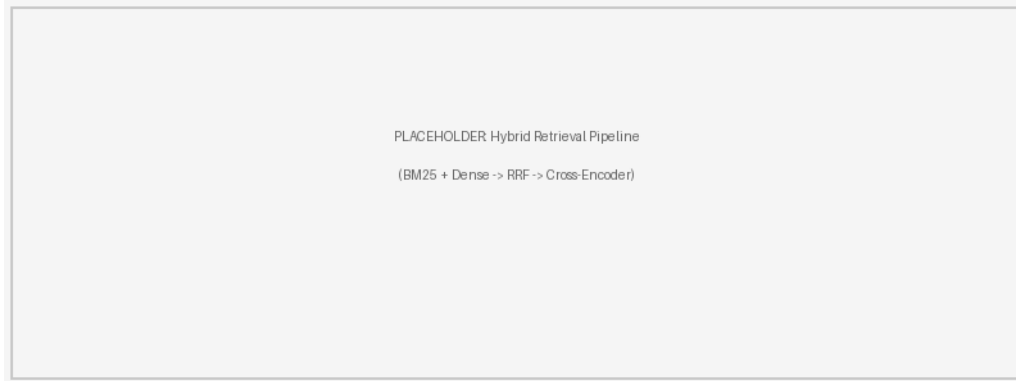


Figure 5.18: [PLACEHOLDER] Hybrid retrieval: BM25 and dense retrieval run in parallel, results are fused via Reciprocal Rank Fusion, then reranked by a cross-encoder. This captures both lexical and semantic matches.

RRF assigns each document a fused score based on its rank in each result list:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \quad (5.13)$$

where R is the set of result lists and k is a constant (typically 60) that dampens the influence of high ranks.

Efficiency at Scale

Matryoshka Representation Learning (MRL)

Traditional embeddings use a fixed number of dimensions. Matryoshka embeddings (Kusupati et al., 2022) train the model so that truncated prefixes of the embedding remain useful: the first 64 dimensions capture coarse semantics, the first 128 add detail, and the full vector provides maximum precision.

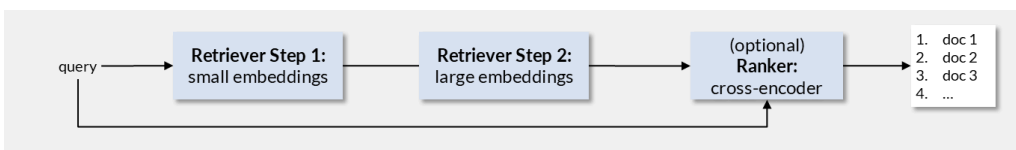


Figure 5.19: Multi-stage retrieval with Matryoshka embeddings: a fast initial retrieval using small (truncated) embeddings narrows candidates, then full-dimension embeddings provide precise reranking. A single model serves both stages.

The training objective adds loss terms at multiple truncation levels:

$$\mathcal{L}_{\text{MRL}} = \sum_{d \in \mathcal{D}} \mathcal{L}_d(\mathbf{x}_{[:d]}, \mathbf{y}_{[:d]}) \quad (5.14)$$

where $\mathcal{D}$ is a set of target dimensions (e.g., {64, 128, 256, 512, 1024}) and $\mathbf{x}_{[:d]}$ denotes the first d components of the embedding.

This enables **multi-stage retrieval within a single model**:

1. Use truncated embeddings (e.g., 128 dimensions) for fast coarse retrieval over the full collection.
2. Use full embeddings (e.g., 1024 dimensions) to rerank the top candidates with higher precision.

Binary and scalar quantization

Storage and comparison costs can be further reduced through quantization:

- **Binary quantization**: replace each float dimension with a single bit (positive arrow.r 1, negative arrow.r 0). Reduces storage by 32x and enables fast Hamming distance computation. Quality loss is typically 5-10% on retrieval benchmarks.
- **Scalar quantization** (int8): map each float to an 8-bit integer. Reduces storage by 4x with minimal quality loss (typically < 2%).

Quantization is most effective as a first-stage filter: use quantized embeddings for coarse candidate retrieval, then rescore with full-precision embeddings.

Practical Considerations

Asymmetric retrieval

Queries are typically short (5-20 tokens) while documents may be long (hundreds to thousands of tokens). This asymmetry means:

- Query embeddings may not capture full intent in limited tokens.
- Document embeddings must compress extensive content into a fixed vector.
- Instruction-tuned models address this with different prefixes for queries vs documents.

Domain adaptation

General-purpose embedding models may underperform on specialized domains (legal, medical, scientific) where vocabulary and language patterns differ from training data. Options for adaptation:

- **Fine-tuning**: train on domain-specific query-document pairs (requires labeled data).
- **Synthetic data**: use LLMs to generate query-document pairs from the target corpus.
- **Domain-specific models**: some domains have specialized embedding models (e.g., PubMedBERT for biomedical text).

Embedding freshness

Unlike inverted indices where adding a document requires only updating postings lists, embedding models may drift: a model trained on 2024 data may produce suboptimal representations for 2026 content discussing new concepts. Re-encoding the full collection with an updated model requires significant compute but ensures consistent representation quality.

Connection to vector search

Dense embeddings create the vectors; the next chapter ([Vector Search](#)) addresses how to efficiently index and search these vectors at scale using approximate nearest neighbor algorithms (HNSW, IVF, product quantization).

Summary

Model Comparison

This chapter presented a progression of methods for representing text semantically. Each generation addressed specific limitations of its predecessors while introducing new tradeoffs:

Method	Representation	Context	Transfer	Key Limitation
LSI (1988)	Dense document vectors via SVD	Global (entire corpus)	Corpus-specific	No inverted index; expensive SVD
Word2Vec (2013)	Dense word vectors	Local window (5-10 words)	Language-level	One vector per word (no polysemy)
GloVe (2014)	Dense word vectors	Global co-occurrence ratios	Language-level	One vector per word (no polysemy)
fastText (2017)	Sub-word vectors	Local window + sub-words	Handles OOV words	Still no contextual meaning
BERT (2019)	Contextualized token vectors	Full sequence (512 tokens)	Task-specific fine-tuning	Poor sentence embeddings without SBERT
Bi-encoder / SBERT (2019)	Single sentence/document vector	Full sequence	Pre-trained, adaptable	No token-level interaction at query time
Cross-encoder (2019)	Relevance score (no vector)	Joint query+document	Task-specific	Too expensive for first-stage retrieval
ColBERT (2020)	Per-token vectors	Full sequence	Pre-trained	Higher storage than bi-encoders
Matryoshka (2022)	Multi-resolution vectors	Full sequence	Pre-trained	Training complexity

The field has converged on a practical consensus: bi-encoders for efficient first-stage retrieval, optionally combined with BM25 for hybrid coverage, followed by cross-encoder reranking for precision-critical applications. ColBERT offers a middle path when single-vector bi-encoders lack sufficient quality but cross-encoder latency is unacceptable.

Key Takeaways

1. **The vocabulary mismatch problem** is the fundamental motivation for semantic search: related terms that share no surface forms cannot be matched by keyword methods alone.
2. **Dimensionality reduction discovers latent structure**: whether via SVD (LSI) or neural training (embeddings), mapping text to a compact space forces the model to group co-occurring patterns into latent topics or semantic directions.
3. **Context transforms meaning**: the same word receives different representations depending on surrounding words in transformer models, solving the polysemy problem that plagued static embeddings.
4. **Bi-encoders enable scalable retrieval**: encoding documents once and comparing via dot product achieves sub-millisecond search over millions of documents. Normalization converts cosine similarity to dot product.
5. **Cross-encoders provide precision**: joint encoding of query and document captures fine-grained token interactions, producing the highest quality relevance scores at the cost of per-pair computation.

6. **Late interaction (ColBERT)** preserves token-level detail while keeping documents pre-computable, achieving 95-98% of cross-encoder quality with manageable latency.
7. **Hybrid retrieval is robust:** combining BM25 (exact match, rare terms) with dense retrieval (semantic matching) via Reciprocal Rank Fusion covers failure modes of either method alone.
8. **Model selection is empirical:** MTEB benchmarks provide standardized comparison, but production performance depends on domain, language, and latency constraints. No single model dominates all use cases.

Key Formulas

LSI document projection:

$$\bar{\mathbf{d}}^\top = \mathbf{d}^\top \mathbf{U}_k \mathbf{S}_k^{-1} \quad (5.15)$$

Maps a term-frequency vector into the k -dimensional latent topic space.

Skip-Gram probability:

$$P(w_s \text{ mid } w_c) = \frac{\exp(\mathbf{u}_s^\top \mathbf{v}_c)}{\sum_{i \in \mathbb{T}} \exp(\mathbf{u}_i^\top \mathbf{v}_c)} \quad (5.16)$$

Probability of context word w_s given center word w_c , parameterized by embedding vectors.

Scaled dot-product attention:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (5.17)$$

Core operation of the transformer: queries select relevant keys, whose associated values are aggregated.

ColBERT MaxSim scoring:

$$\text{score}(q, d) = \sum_{i=1}^{|q|} \max_{j=1}^{|d|} \mathbf{q}_i^\top \mathbf{d}_j \quad (5.18)$$

Each query token's best document-token match is summed for the final relevance score.

Self-Check Questions

1. **(Understand)** Why does truncating the SVD to k dimensions bring semantically related terms closer together, even if they never co-occur in the same document?
2. **(Understand)** Explain why Word2Vec assigns the same embedding to “bank” regardless of whether it appears in a financial or geographic context. What architectural change in transformers solves this?
3. **(Analyze)** A bi-encoder encodes a 500-word document into a single 768-dimensional vector. What information is necessarily lost compared to the per-token representations? Give a concrete example of a query that might fail because of this compression.
4. **(Analyze)** Compare the computational cost of ranking 10,000 documents using: (a) a bi-encoder, (b) a cross-encoder, (c) ColBERT with MaxSim. Express each in terms of the number of transformer forward passes required.
5. **(Evaluate)** Your system uses BM25 retrieval followed by cross-encoder reranking with $k = 100$. A user reports that relevant documents for the query “ML model deployment best practices” are not appearing in results, despite existing in the collection. Diagnose the most likely failure point and propose a fix.
6. **(Evaluate)** You are designing a semantic search system for a multilingual legal corpus (5 million documents, 12 languages). Justify your choice of embedding model, chunking strategy, and retrieval architecture.
7. **(Apply)** Given Matryoshka embeddings with dimensions in {64, 128, 256, 768}, design a two-stage retrieval pipeline. Specify which dimension you use at each stage and why.

Further Reading

- Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). **Indexing by Latent Semantic Analysis**. *Journal of the American Society for Information Science*, 41(6), 391-407. [PDF](#) The foundational paper introducing LSI and demonstrating that SVD on term-document matrices captures latent semantic structure.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). **Distributed Representations of Words and Phrases and their Compositionality**. *Advances in Neural Information Processing Systems (NeurIPS)*. [arXiv:1310.4546](#) Introduces negative sampling and phrase-level Skip-Gram, making Word2Vec practical at scale.
- Reimers, N. & Gurevych, I. (2019). **Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks**. *Proceedings of EMNLP-IJCNLP 2019*. [arXiv:1908.10084](#) The paper that made transformer-based sentence embeddings practical for retrieval by introducing the bi-encoder training framework.
- Khattab, O. & Zaharia, M. (2020). **ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT**. *Proceedings of SIGIR 2020*. [arXiv:2004.12832](#) Introduces the late interaction paradigm (MaxSim), achieving near cross-encoder quality with much lower latency.
- Kusupati, A., Bhatt, G., Rber, A., et al. (2022). **Matryoshka Representation Learning**. *Advances in Neural Information Processing Systems (NeurIPS 2022)*. [arXiv:2205.13147](#) Trains embedding models to produce useful representations at any truncation level, enabling multi-resolution retrieval.
- Muennighoff, N., Tazi, N., Magne, L., & Reimers, N. (2023). **MTEB: Massive Text Embedding Benchmark**. *Proceedings of EACL 2023*. [arXiv:2210.07316](#) Establishes the standard benchmark for evaluating embedding models across retrieval, similarity, classification, and clustering tasks.